Լաբորատոր աշխատանք 10

Դասեր C++ լեզվում · «Բանկային հաշվի գործարքներ» · N14

Կատարող՝ Hakob Avagyan · 2026

Աշխատանքի նպատակ

Ամրապնդել դասի նախագծման հմտությունները՝ մասնավոր (`private`) տվյալներ, կոնստրուկտոր, `const` մեթոդներ և գործարքների զանգվածի մշակում՝ մնացորդի հաշվարկի օրինակով։

Խնդրի պահանջը

Ստեղծել `AccountLedger` դաս, որը պահում է գործարքների գումարների զանգվածը և `p` սկզբնական մնացորդը։ Դրական գումարը մուտք է, բացասականը՝ ելք։ Հաշվել՝

- վերջնական մնացորդը,
- գործարքից հետո ստացված ամենափոքր մնացորդի ինդեքսը,
- այն գործարքների քանակը, որոնցից հետո մնացորդը բացասական է։

Տեսական մաս

Դասի դաշտերը (`start`, `tx[]`, `count`) գտնվում են `private` բաժնում, ուստի փոփոխվում են միայն `add()` մեթոդով, որը մերժում է զրոյական գումարը և զանգվածի գերլցումը։ Հաշվարկային մեթոդները `finalBalance`, `minBalanceIndex` և `negativeCount` նշված են `const`, քանի որ օբյեկտը չեն փոփոխում։ Մնացորդը յուրաքանչյուր գործարքից հետո հաշվվում է կուտակմամբ՝ `b += tx[i]`, այնպես որ բոլոր երեք արդյունքները ստացվում են զանգվածի մեկ անցումով։

Ամենափոքր մնացորդի որոնման մեջ առաջին գործարքի մնացորդը ընդունվում է սկզբնական նվազագույն, հետագա արժեքները համեմատվում են խիստ «`<`» նշանով, ուստի հավասարության դեպքում վերադարձվում է **ամենավաղ** ինդեքսը։ Սկզբնական մնացորդ `p`-ն որպես թեկնածու չի դիտարկվում, քանի որ պահանջվում է մնացորդը *գործարքից հետո*։ Առանց գործարքների մեթոդը վերադարձնում է `-1`։ Ինդեքսը հաշվվում է 0-ից։

Դասի մեթոդները

Մեթոդ	Նշանակություն
`AccountLedger(double p)`	կոնստրուկտոր՝ սահմանում է սկզբնական մնացորդը, գործարքների քանակը՝ 0
`bool add(double)`	գործարքի ավելացում (գումար ≠ 0, առավելագույնը 20 գործարք)
`double finalBalance() const`	վերջնական մնացորդ՝ p + Σ գործարքներ
`int minBalanceIndex(double&) const`	գործարքից հետո ամենափոքր մնացորդի ինդեքսը (և արժեքը հղումով) կամ −1
`int negativeCount() const`	բացասական մնացորդով գործարքների քանակ

Ծրագրի կոդը

ledger.cpp

```cpp
// Լաբորատոր աշխատանք 10 — AccountLedger դաս (բանկային հաշվի գործարքներ)
#include <iostream>
#include <iomanip>
#include <limits>
#include <cstdlib>
#ifdef _WIN32
#include <windows.h>
#endif
using namespace std;

const int MAX_TX = 20;

class AccountLedger {
    double start;            // p — սկզբնական մնացորդ
    double tx[MAX_TX];       // գործարքներ (+ մուտք, − ելք)
    int count;
public:
    AccountLedger(double p) : start(p), count(0) {}

    bool add(double amount) {            // false՝ գումար 0 կամ զանգվածը լիքն է
        if (count >= MAX_TX || amount == 0) return false;
        tx[count++] = amount;
        return true;
    }
    double finalBalance() const {
        double b = start;
        for (int i = 0; i < count; i++) b += tx[i];
        return b;
    }
    // Գործարքից հետո ամենափոքր մնացորդի ինդեքսը (0-ից), առանց գործարքների՝ -1
    int minBalanceIndex(double& minB) const {
        int best = -1;
        double b = start;
        for (int i = 0; i < count; i++) {
            b += tx[i];
            if (best == -1 || b < minB) { minB = b; best = i; }
        }
        return best;
    }
    // Քանի գործարքից հետո է մնացորդը բացասական
    int negativeCount() const {
        int k = 0;
        double b = start;
        for (int i = 0; i < count; i++) { b += tx[i]; if (b < 0) k++; }
        return k;
    }
};

// Թվի մուտքագրում՝ սխալի դեպքում կրկնվում է
double readNumber(const char* prompt, double lo, double hi, bool nonZero) {
    double v;
    while (true) {
        cout << prompt;
        if (cin >> v && v >= lo && v <= hi && !(nonZero && v == 0)) return v;
        if (cin.eof()) exit(1);
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        cout << "  Սխալ մուտքագրում, կրկնեք\n";
    }
}

int main() {
#ifdef _WIN32
    SetConsoleOutputCP(CP_UTF8);
```

```
    SetConsoleCP(CP_UTF8);
#endif
    cout << fixed << setprecision(2);
    double p = readNumber("Սկզբնական մնացորդ p: ", -1e9, 1e9, false);
    int n = (int)readNumber("Գործարքների քանակ (1-20): ", 1, MAX_TX, false);

    AccountLedger acc(p);
    for (int i = 0; i < n; i++)
        acc.add(readNumber("Գործարք (+ մուտք, - ելք): ", -1e9, 1e9, true));

    double minB = 0;
    int m = acc.minBalanceIndex(minB);
    cout << "\nՎերջնական մնացորդ: " << acc.finalBalance()
         << "\nԱմենափոքր մնացորդի ինդեքս: " << m << " (մնացորդ " << minB << ")"
         << "\nԲացասական մնացորդով գործարքներ: " << acc.negativeCount() << '\n';
    return 0;
}
```

Օրինակային մուտք և ելք

p = 100, 7 գործարք։ Մուտքում միտումնավոր ներառված են սխալ արժեքներ (գործարքների քանակ 0, տառեր, գումար 0)՝ ստուգումները ցույց տալու համար։ Կանաչով նշված է օգտատիրոջ մուտքը։

```
Terminal — g++ -std=c++17 -Wall -Wextra ledger.cpp -o ledger && ./ledger

Սկզբնական մնացորդ p: 100
Գործարքների քանակ (1-20): 0
  Սխալ մուտքագրում, կրկնեք
Գործարքների քանակ (1-20): 7
Գործարք (+ մուտք, - ելք): abc
  Սխալ մուտքագրում, կրկնեք
Գործարք (+ մուտք, - ելք): 50
Գործարք (+ մուտք, - ելք): -200
Գործարք (+ մուտք, - ելք): 0
  Սխալ մուտքագրում, կրկնեք
Գործարք (+ մուտք, - ելք): 30
Գործարք (+ մուտք, - ելք): -20
Գործարք (+ մուտք, - ելք): 150
Գործարք (+ մուտք, - ելք): -300
Գործարք (+ մուտք, - ելք): 40

Վերջնական մնացորդ: -150.00
Ամենափոքր մնացորդի ինդեքս: 5 (մնացորդ -190.00)
Բացասական մնացորդով գործարքներ: 5
```

Ձեռքով ստուգում

Գործարք	1	2	3	4	5	6	7
Գումար	+50	−200	+30	−20	+150	−300	+40
Մնացորդ (p = 100)	150	−50	−20	−40	110	−190	−150

Վերջնական մնացորդ՝ **−150**։ Ամենափոքրը՝ −190՝ 6-րդ գործարքից հետո (ինդեքս **5**)։ Բացասական մնացորդ՝ 2, 3, 4, 6, 7-րդ գործարքներից հետո՝ **5** գործարք։ Արդյունքը համընկնում է ծրագրի ելքի հետ։

Հանձնման ստուգաթերթ

Կետ	Արդյունք
Ծրագիրը կոմպիլացվում է առանց զգուշացումների	✔ g++ -Wall -Wextra
Տվյալները դասի `private` բաժնում են	✔ հասանելի են միայն մեթոդներով
Հաշվարկային մեթոդները `const` են	✔ 4 մեթոդ
Մուտքային տվյալների ստուգում	✔ քանակ 1–20, գումար ≠ 0, թվային ձևաչափ
Դատարկ գործարքների դեպքը	✔ minBalanceIndex → −1

Եզրակացություն

Ստեղծվեց `AccountLedger` դասը, որը պահում է սկզբնական մնացորդը և գործարքների զանգվածը։ Օրինակում (p = 100, 7 գործարք) վերջնական մնացորդը −150 է, ամենափոքր մնացորդը −190՝ 6-րդ գործարքից հետո (ինդեքս 5), բացասական մնացորդով գործարքների քանակը՝ 5։ Արդյունքները համընկնում են ձեռքով հաշվարկի հետ։